

Esame

lunedì 3 marzo 2025 22:50

Composto di 3 parti tutte obbligatorie:

- Scritto
- Test domande a risposta multipla
- Esercizio/Homework 1
- Esercizio/Homework 2

Esame Scritto

- Compito a quiz da fare direttamente al computer in laboratorio sulla piattaforma elearning.uniroma1.it. Domande a risposta chiusa; una sola opzione vera.
- Ci saranno un certo numero di domande da fare in poco tempo!! 30 minuti per 30/40 domande (dettagli verranno forniti prima dell'esame).
- Nel caso in cui ci siano più iscritti all'esame che posti in laboratorio, l'esame si farà in più turni, comunicati sul sito del corso
- Voto in trentesimi

Homework (1 e 2)

- Da svolgere in gruppo 2 - 3 studenti (gruppi singoli ammessi solo per studenti con status lavoratore)
- Nel programma del corso sono allocate 9 ore per homework 1 e 12 ore per homework 2
- Da consegnare prima dell'esame scritto (vedi scadenze). Dovrà essere consegnato codice e relazione (dettagli verranno forniti con il testo degli homework)
- verificato mediante presentazione di tutti i membri del gruppo
- voto in trentesimi (un voto per ogni homework)

Voto Finale = MEDIA(Voto Scritto + Voto Homework 1 + Voto Homework 2)

Gli homework vanno consegnati prima di svolgere l'esame scritto.

Le scadenze per la consegna degli homework sono le seguenti:

- 3/06/2024 (21:00) per svolgere l'esame il 9-10/06/2024
- 4/07/2024 (21:00) per svolgere l'esame il 11-14/07/2024
- 2/09/2024 per svolgere l'esame il 9/09/2024
- **in tutte le sessioni a seguire (aka Sessione invernale e Sessioni straordinarie)** 5 giorni lavorativi antecedenti la data di esame

Regole Verbalizzazione Esami

- Secondo Modulo SO (SOII) non va verbalizzato direttamente ma confluisce in un'unica verbalizzazione con il Primo Modulo (SOI). $SOI + SOII = SO$ (12 crediti)
- Per canale M-Z verbalizza Prof. FABIO DE GASPARI - notifico io il superamento di SOII. Non dovete mandare voi studenti mai al prof. De Gasperi.
- if (Voto_SOI $\geq$ 18 AND Voto_SOII $\geq$ 18) **Voto_SO** = (Voto_SOI + Voto_SOII) / 2;
- **Voti SOI e SOII devono essere conseguiti nell'arco temporale di un anno solare dal superamento del primo dei due moduli.**

MyFileTransferApp

venerdì 21 febbraio 2025 15:25

Lo studente deve realizzare un applicazione client-server che permette di trasferire file da un client ad un server (scrittura di un file) e, viceversa, da un server ad un client (lettura di un file).

L'applicazione è composta da due programmi, un client che effettua richieste di lettura e scrittura di un file, un server che riceve le richieste e gestisce le richieste dei client.

Assumendo che l'applicazione server si chiama myFTserver e l'applicazione client si chiama myFTclient il comportamento deve essere il seguente.

Server

Il comando

```
myFTserver -a server_address -p server_port -d ft_root_directory
```

esegue il programma server mettendolo in ascolto su di un determinato indirizzo IP e porta ed indicando la directory nella quale andare a scrivere/leggere i file. Se ft_root_directory non esiste deve essere creata.

Una volta in esecuzione, il server deve accettare connessioni da uno o più client e gestirle concorrentemente.

Richieste di scrittura concorrenti sullo stesso file devono essere opportunamente gestite (come la richiesta di creazione concorrente di path con lo stesso nome).

Il programma server deve gestire tutte le eccezioni come ad esempio: richiesta di accesso a file non esistente (per la lettura), errore nel binding su IP e porta, parametri di invocazione del comando errati o mancanti, spazio su disco esaurito, interruzione della connessione con il client.

Client

Il comando

```
myFTclient -w -a server_address -p port -f local_path/filename_local -o remote_path/filename_remote
```

esegue il programma client, crea una connessione con il server specificato da server_address:port, e scrive il file local_path/filename_local sul server con nome filename_remote e nella directory specificata da remote_path. remote_path avrà root nella directory del server specificata con ft_root_directory

il comando

```
myFTclient -w -a server_address -p port -f local_path/filename_local
```

si comporta come il precedente ma il nome del path remoto e del file remoto sono gli stessi del path e file locale.

il comando

```
myFTclient -r -a server_address -p port -f remote_path/filename_remote -o local_path/filename_local
```

esegue il programma client, crea una connessione con il server specificato da server_address:port e legge il file specificato da remote_path/filename_remote trasferendolo al programma client che lo scriverà nella directory local_path assegnando il nome filename_local

il comando

```
myFTclient -r -a server_address -p port -f remote_path/filename_remote
```

si comporta come il precedente ma il nome del path locale e del file locale sono gli stessi del path e file remoto.

il comando

```
myFTclient -l -a server_address -p port -f remote_path/
```

permette al client di ottenere la lista dei file che si trovano in remote_path (effettua sostanzialmente un ls -la remoto). la lista dei file deve essere visualizzata sullo standard output del terminale da cui viene eseguito il programma myFTclient.

Il programma client deve gestire tutte le eccezioni del caso. come ad esempio: parametri di input errati, file remoto non esistente (lettura), spazio di archiviazione insufficiente sul server (scrittura) e sul client, interruzione della connessione con il server

Valutazione:

Il codice sottomesso deve essere commentato in modo chiaro ed estensivo
il codice deve essere accompagnato da una relazione (2 pagine massimo) che descriva le scelte operate.
In sede di verifica, la studentessa/lo studente deve dimostrare che il codice funzioni

La consegna deve avvenire come segue:

Sottomettere un archivio .tgz o .zip, nominato come MATRICOLA_HW2 contenente i file .h, .c e .pdf (per la relazione)

MyFinger

venerdì 21 febbraio 2025 15:30

L'homework consiste nel creare un programma che ha lo stesso comportamento del comando `finger[1]`

Non e' richiesta l'implementazione del lookup delle informazioni per utenti remoti
Nell'implementazione non puo' essere utilizzata la famiglia di funzioni `exec*(3)`

Il codice deve essere organizzato in almeno un file header (.h) che contiene tutte le dichiarazioni di variabili, costanti, e prototipi di funzioni) ed un file .c che contiene il codice delle funzioni progettate ed implementate.

Valutazione:

Il codice sottomesso deve essere commentato in modo chiaro ed estensivo
il codice deve essere accompagnato da una relazione (2 pagine massimo) che descriva le scelte operate.

In sede di verifica, la studentessa/lo studente deve dimostrare che il codice funzioni

La consegna deve avvenire come segue:

Sottomettere un archivio .tgz o .zip, nominato come `MATRICOLA_HW1` contenente i file .h, .c e .pdf (per la relazione)

Riferimenti

[1] `finger(1)` - Linux man page <https://linux.die.net/man/1/finger>

Shell e Comandi

lunedì 24 febbraio 2025 11:14

La **Shell** o **Terminal** è un interprete di comandi. Esistono vari tipi di shell: sh, bash, ksh, ecc. Negli appunti useremo **bash**. Il prompt tipico in bash è:

nomeuser@nomemacchina:~cammino\$ comando

Dove cammino è il path dalla **directory home** alla directory attuale. Se c'è solo ~ allora si è nella home, se ~ manca allora cammino è il **path assoluto (root)**

```
fra@Lenovo:~$  
fra@Lenovo:~$ cd Desktop  
fra@Lenovo:~/Desktop$ cd /  
fra@Lenovo:/$ ls  
bin          home          mnt          sbin.usr-is-merged  tmp  
bin.usr-is-merged  lib          opt          snap              usr  
boot         lib64        proc         srv              var  
cdrom        lib.usr-is-merged  root        swapfile  
dev          lost+found   run         sys  
etc          media       sbin        timeshift  
fra@Lenovo:/$ cd bin  
fra@Lenovo:/bin$
```

Comandi

Ogni comando ha questa forma:

comando [opzioni] argomentiobbligatori

Deve esserci almeno un argomento, ma possono essercene anche **più d'uno** se {argomento}.

es: cp [-r] [-i] [-a] [-u] {filesorgenti} filedestinazione

Ci possono essere 0,1 o più argomenti opzionali se [opzione...]

es: [opzioni] [pid...]

A volte bisogna separare gli argomenti in base al carattere indicato dal comando se [carattere opzione...]

es: chmod mode[, mode...] filename

Le opzioni sono tipicamente composte da:

- carattere (vecchia versione) (es: -r, -i)
- parola (nuova versione) (es: --recursive, --interactive)

Le opzioni possono avere un **argomento**:

-k1, -k 1, --key=1

Le opzioni senza argomento sono **raggruppabili**

-b -r -c è equivalente a -brc

Esistono anche opzioni BSD-style che non usano il dash

Es: tar xzf nomefile.tgz

Un comando utile per conoscere i comandi linux è il comando **man** che mostra i manuali di riferimento per il comando indicato

man [sezione] comando

Es man man

```
MAN(1)                                Manual pager utils                                MAN(1)  
  
NAME  
man - an interface to the system reference manuals  
  
SYNOPSIS  
man [man options] [[section] page ...] ...  
man -k [apropos options] regexp ...  
man -K [man options] [section] term ...  
man -f [what's options] page ...  
man -l [man options] file ...  
man -w|-W [man options] page ...  
  
DESCRIPTION  
man is the system's manual pager. Each page argument given to man is normally the name of a  
program, utility or function. The manual page associated with each of these arguments is  
then found and displayed. A section, if provided, will direct man to look only in that sec-  
tion of the manual. The default action is to search in all of the available sections follow-  
ing a pre-defined order (see DEFAULTS), and to show only the first page found, even if page  
exists in several sections.  
  
The table below shows the section numbers of the manual followed by the types of pages they  
contain.  
  
1 Executable programs or shell commands  
2 System calls (functions provided by the kernel)
```

Es man ls

```
LS(1)                                User Commands                                LS(1)  
  
NAME  
ls - list directory contents  
  
SYNOPSIS  
ls [OPTION]... [FILE]...  
  
DESCRIPTION  
List information about the FILES (the current directory by default). Sort entries alphabeti-  
cally if none of -cftuvSUX nor --sort is specified.  
  
Mandatory arguments to long options are mandatory for short options too.  
  
-a, --all  
do not ignore entries starting with .  
  
-A, --almost-all  
do not list implied . and ..  
  
--author  
with -l, print the author of each file  
  
-b, --escape  
print C-style escapes for nongraphic characters
```

Oppure si può usare

comando --help

per avere una definizione ridotta per l'uso del comando

Esercizio: perché

ps -p "\$\$" -ocmd -h

restituisce "bash" come output

Usando man ps vediamo che:

- p: seleziona in base all'id del processo
Con -p "\$\$" stiamo cercando il processo con l'id \$\$ (che rappresenta il processo corrente, quindi il terminale bash)
- o format oppure -ofomat: esegue l'output delle colonne specifiche definite nel formato
Con -ocmd (equivalente a -o cmd) stiamo dicendo di eseguire l'output solo della colonna CMD (il nome semplice del processo) dei processi trovati
- h: rimuove gli header delle colonne

```
fra@Lenovo:~$ ps -p "$$"
  PID TTY          TIME CMD
 14623 pts/1    00:00:00 bash
fra@Lenovo:~$ ps -p "$$" -ocmd
CMD
bash
fra@Lenovo:~$ ps -p "$$" -o cmd -h
bash
fra@Lenovo:~$ echo $0
bash
```

Utenti e Gruppi

All'installazione di linux bisogna specificare almeno un **utente principale**. Nelle macchine virtuali già pronte l'utente è già preimpostato

In Linux esistono:

- **Utenti reali:** persone vere che hanno una cartella personale e possono entrare nel sistema
- **Utenti di sistema:** "utenti" creati per far girare programmi specifici (es server web, database, ecc) che non hanno una password e non possono "entrare" nel computer. Esistono solo perche **ogni processo** deve **appartenere a un utente**

Neanche **root**, che è il superutente che può eseguire ogni comando, può eseguire il login (per questioni di sicurezza). Però un utente può **acquisire i diritti di root** col comando **su** e **sudo**

Ogni utente deve **appartenere** ad un **gruppo**. Esistono:

- **Gruppo primario (usergroups):** quando si crea l'utente "mario", linux crea un gruppo chiamato "mario" usato per gestire i permessi dei file creati da mario
- **Gruppi amministrativi:** esistono gruppi speciali (es sudo, wheel, audio, ecc) che dividono i diritti per usare certi comandi. Se aggiungiamo mario al gruppo sudo, mario acquisirà il diritto di usare il comando sudo.

Il comando **groups [nomeutente...]** permette di vedere i gruppi a cui appartengono gli utenti dati in argomento

Sudo

L'utente principale è un **sudoer**, cioè appartiene al gruppo predefinito **sudo**. Gli utenti sudoer possono eseguire comandi da **super utente (root)** usando **sudo comando**

sudo è un comando che prende un comando e chiede la password dell'utente per eseguire da root il comando dato

Creazione di Utenti

Per creare un nuovo utente (di default non appartiene al gruppo sudo):

```
adduser nuovoutente
```

Per aggiungere l'utente ad un gruppo:

```
adduser utente gruppo
```

Per rimuovere l'utente:

```
sudo userdel utente
```

(con l'opzione -r si elimina anche la sua home directory)

FileSystem (fs)

giovedì 12 marzo 2026 15:54

Il **filesystem** è un'organizzazione di un'area di memoria basata sul concetto di file e di directory con una **struttura gerarchica ad albero**.

I file **regolari** contengono sequenze di bit dell'area di memoria sulla quale c'è il filesystem. File **non regolari o speciali** modellano unità di I/O a blocchi/caratteri

Linux ha un **solo filesystem principale** che ha come directory radice **/ (root)**. Tutti i file e directory sono contenuti in **/**

Ogni file o dir è raggiungibile da **/** mediante un **path assoluto**. Possiamo usare **~** per indicare il path **/home/userX**

/home/fra/dir1/dir11/dir2/file.pdf

~/dir1/dir11/dir2/file.pdf

Current Working Directory (cwd)

Per conoscere la **directory corrente** (cwd) si può usare il comando **pwd**

Per cambiare cwd si usa il comando **cd [path]**

- **[path]** può essere assoluto (da **/**) o relativo (da **~**)
- **cd** senza path ritorna alla **home** (**~**)
- nel path si può usare **..** (torna indietro alla parent dir) o **.** (va alla dir attuale)

```
fra@Lenovo:~$ cd Pictures/test
fra@Lenovo:~/Pictures/test$ pwd
/home/fra/Pictures/test
fra@Lenovo:~/Pictures/test$ cd ..
fra@Lenovo:~/Pictures$ cd .
fra@Lenovo:~/Pictures$ cd
fra@Lenovo:~$ cd /home/fra/Pictures/test
fra@Lenovo:~/Pictures/test$ cd
fra@Lenovo:~$ cd Pictures/../Pictures/test
fra@Lenovo:~/Pictures/test$ cd ../../../Pictures../test
fra@Lenovo:~/Pictures/test$
```

Contenuto di una Directory

Il comando **ls** restituisce la lista dei contenuti di una directory

- **ls**: mostra il contenuto della cwd
- **ls nomedir**: mostra il contenuto della directory cwd/nomedir
- **ls nomefile**: restituisce il nome del file

Esistono anche file **"nascosti"** tipicamente di configurazione o usati a supporto di comandi e applicazioni.

Per vedere i tutti i file di una directory si usa **ls [-a | --all]**

Es: **.bash_history** contiene tutta la history del terminale

Per visualizzare ricorsivamente anche il contenuto delle **sottodirectory** possiamo usare **ls [-R | --recursive]**

Si può anche visualizzare l'**albero delle directory** con

tree [-a] [-L maxdepth] [-d] [-x] [nomedir]

- **-a**: mostra tutti i file (anche quelli nascosti)
- **-L maxdepth**: profondità massima dell'albero
- **-d**: mostra solo le directory
- **-x**: rimane sul file-system corrente

(ci sono molte altre opzioni per il comando)

```
LISTING OPTIONS
-a      All files are printed. By default tree does not print hidden files (those
        beginning with a dot '.'). In no event does tree print the file system con-
        structs '.' (current directory) and '..' (previous directory).

-d      List directories only.

-l      Follows symbolic links if they point to directories, as if they were directo-
        ries. Symbolic links that will result in recursion are avoided when detected.

-f      Prints the full path prefix for each file.

-x      Stay on the current file-system only. Ala find -xdev.

-L level
        Max display depth of the directory tree.

-R      Recursively cross down the tree each level directories (see -L option), and
        at each level outputting to a file named 00Tree.html (ala -o).

-P pattern
        List only those files that match the wild-card pattern. You may have multi-
```

```

fra@Lenovo:~$ tree -a -L 3
.
├── .bash_history
├── .bash_logout
├── .bashrc
├── .cache
│   ├── cinnamon
│   │   ├── spices
│   │   └── cinnamon-control-center
│   │       └── wacom
│   ├── cs_themes
│   │   └── icons
│   └── evolution
│       ├── addressbook
│       ├── calendar
│       ├── mail
│       ├── memos
│       ├── sources
│       └── tasks
├── flatpak
├── ld.so
├── system-cache
└── fontconfig
    └── 0bd3dc0958fa2205aaaa8ebb13e2872b-

fra@Lenovo:~$ tree -d -L 3
.
├── Desktop
│   ├── AutoHotKey
│   ├── cpp
│   │   ├── 2025-01-09
│   │   ├── 2025-02-05
│   │   ├── 2025-03-21
│   │   ├── 2025-06-12
│   │   ├── 2025-07-08
│   │   ├── 2025-09-08
│   │   ├── 2025-11-03
│   │   └── 2026-01-14
│   ├── drive-download-20260119T150746Z-1-001
│   └── ProgrWeb
│       ├── cmd
│       ├── demo
│       ├── doc
│       ├── images
│       ├── node_modules
│       ├── service
│       ├── vendor
│       └── webui
└── Test

```

Creazione di Directory e File

`mkdir [-p] nomedir`

- Crea una directory **nomedir** **vuota**
- Per creare un intero path di directory bisogna usare l'opzione **-p**
Es: `mkdir dir1/dir2/dir3` creerà 3 directory una dopo l'altra

`touch nomefile`

- Crea un file **nomefile** **vuoto**
- Per editare il contenuto di un file si può usare **gedit**, **nano**, **pico**, **vi** (personalmente io uso **micro**, è da installare)
gedit nomefile & permette di editare il file in una finestra esterna al terminale così da poter usare il terminale nel mentre

Mounting

Il filesystem root (/) contiene elementi **eterogenei** (disco interno, usb esterna, filesystem di rete, filesystem virtuali, ecc)

Tutto questo è possibile grazie al meccanismo di **mounting**:

Una qualsiasi dir D può diventare il **punto di mount** per un altro filesystem F se e solo se la dir root di F diventa accessibile da D

- Se D è vuota, dopo il mount conterrà F
- Se D non è vuota, dopo il mount conterrà F ma i dati in D non vengono rimossi ma saranno accessibili dopo l'unmount di F

Partizioni

Un disco può essere **suddiviso** in due o più **partizioni** divise tra loro

Una partizione **A** può contenere l'**OS** (montata su /) e la partizione **B** i dati degli **utenti** (montata su **/home**)

In questo modo possiamo reinstallare l'OS su A (o installare una versione diversa di Linux in una partizione C) senza toccare i dati degli utenti in B che monterò in /home sul nuovo fs

Nome	Journal	Partiz (TB)	File (TB)	Nome file (bytes)
Ext2	No	32	2	255
Ext3	Sì	32	2	255
Ext4	Sì	1000	16	255
ReiserFS	Sì	16	8	4032

Tipi di FileSystem

Ci sono filesystem non linux, come quelli di Windows (NTFS, MSDOS, FAT32, EXFAT) che possono essere montati su di un fs Linux

Il comando **mount** permette di montare fs e vedere gli fs montati

- **cat /etc/mtab**: visualizza fs montati
- **cat /etc/fstab**: visualizza fs montati al boot

(**cat nomefile** è un comando per stampare nel terminale un file di testo)

Directory	Spiegazione	Montata
/boot	Kernel e file di boot	NO
/bin	Binari (programmi eseguibili) di base	NO
/dev	Devices (periferiche) hardware e virtuali	boot
/etc	File di configurazione di sistema	NO
/proc	Dati e statistiche dei processi e parametri del kernel	boot
/sys	Informazioni e statistiche di device di sistema	boot
/media	Mountpoint per device di I/O (es: CD, DVD, USB pen)	quando necessario
/mnt	(come /media)	quando necessario
/sbin	Binari di sistema	NO
/var	File variabili (log file, code di stampa, mail ...)	NO
/tmp	File temporanei	NO
/lib	Librerie	NO

Tipiche directory di primo livello

File passwd e group

passwd e **group** sono due file di testo con una struttura definita e conosciuta dai programmi che interagiscono con quei file.

Sono **organizzati per righe** dove ogni riga contiene vari campi separati da : e termina con **line feed LF (0x0A)**

/etc/passwd: contiene tutti gli utenti. Struttura:

username:password:uid:gid:gecos:homedir:shell

- uid = userID, gid = groupID
- Al posto della password ci può essere una **x** (cifrata)
- Normalmente il primo utente ha UID 1000 e GID 1000

/etc/group: contiene tutti i gruppi. Struttura:

groupname:password:gid:lista utenti

- Gli utenti nella lista sono separati da una virgola

```
fra@Lenovo:~/Desktop$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:./usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:./usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,./usr/lib/dhcpcd:/bin/false
messagebus:x:101:101:./nonexistent:/usr/sbin/nologin
syslog:x:102:102:./nonexistent:/usr/sbin/nologin

fra@Lenovo:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog, fra
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
fax:x:21:
voice:x:22:
cdrom:x:24:fra
floppy:x:25:
tape:x:26:
sudo:x:27:fra
audio:x:29:
dip:x:30:fra
```

File

giovedì 12 marzo 2026 18:23

Ogni file (anche le dir sono file) nel fs è rappresentato da una struttura dati **inode**, univocamente identificata da un **inode number**

La **cancellazione** di un file **libera l'inode number** che verrà riusato quando necessario per un nuovo file

Principali Attributi

- **Type**: tipo di file (regular, block, fifo, ecc)
- **User ID**: id dell'utente proprietario del file
- **Group ID**: id del gruppo a cui è associato il file
- **Mode**: **permessi** (read, write, exec) di accesso per il proprietario, il gruppo e altri
- **Size**: dimensione in byte del file
- **Timestamps**:
 - **ctime** (inode changing time): cambiamento di un attributo
 - **mtime** (content modification time): solo scrittura
 - **atime** (content access time): solo lettura
- **Link count**: numero di **hard links**
- **Data pointers**: puntatore alla lista dei blocchi che compongono il file;
Se si tratta di una directory, il contenuto su disco è costituito di una tabella con 2 colonne: nome file/dir e il suo inode number

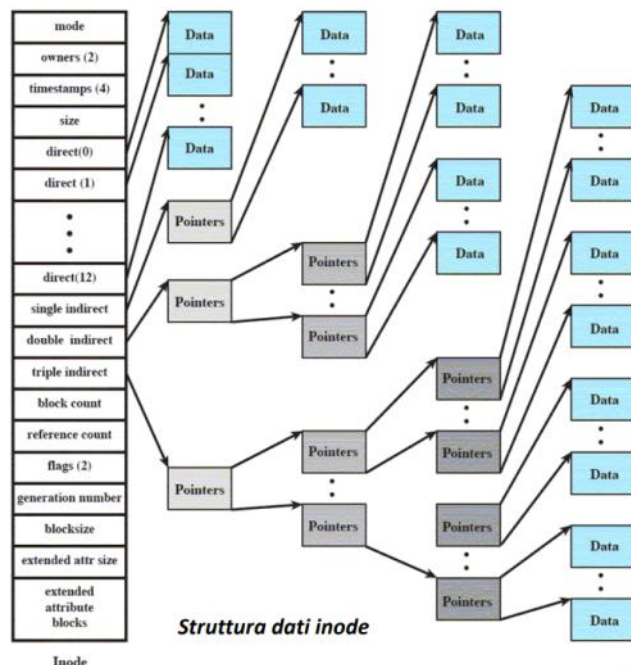
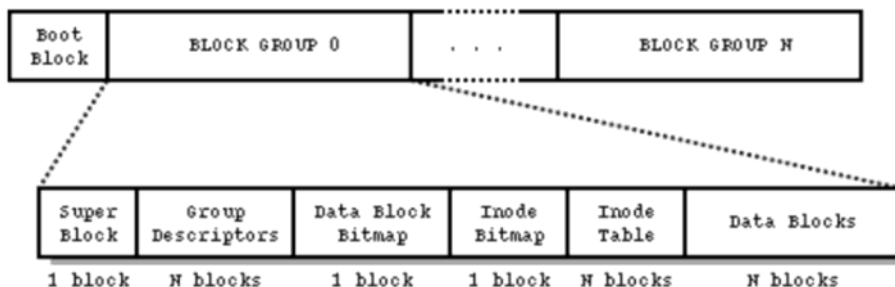
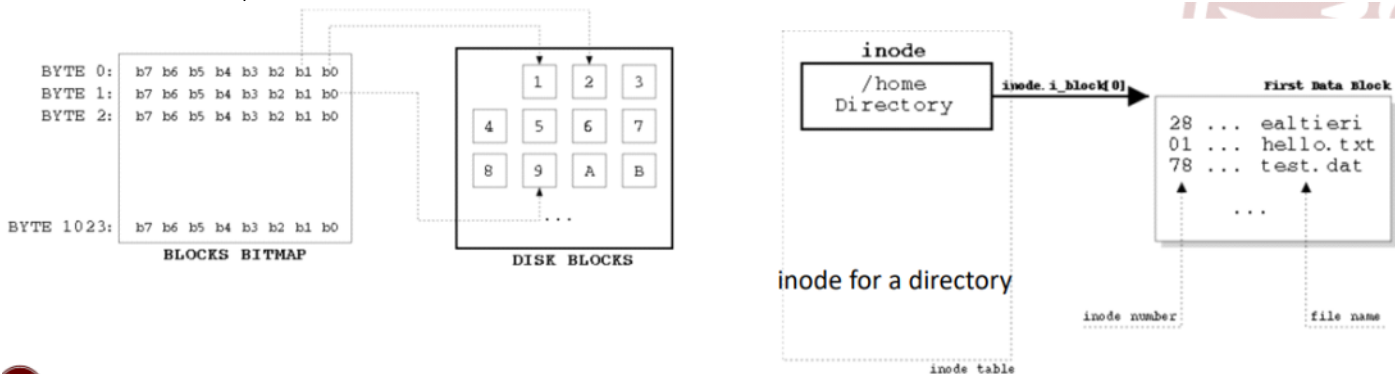


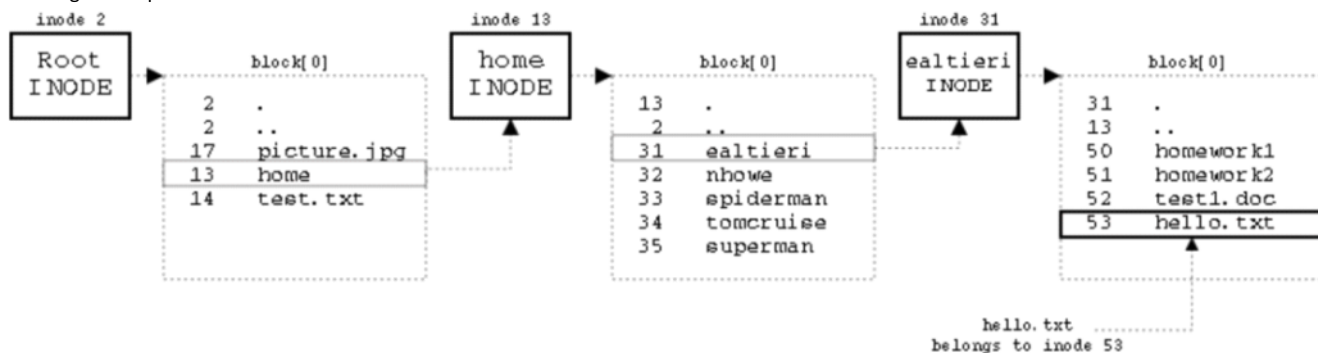
Tabella Inode



Si trova all'inizio del disco. Es per ext2:



Come viene seguito un path:



Per vedere le informazioni dell'inode di un file possiamo usare il comando ls:

- **opzione -i**: stampa l'inode number
- **opzione -l**: mostra diritti, user, group, size, date e time

- **ls -l nomefile**: mostra le informazioni del singolo file
- **ls -l nomedirectory**: mostra le informazioni di tutti i file nella cartella
- **ls -n nomefile/nomedir**: mostra l'uid e gid invece del nome esteso
- Per vedere **timestamp**:
 - Senza niente: mtime
 - **-l -c (oppure -lc)**: ctime
 - **-l -u (oppure -lu)**: atime

La **dimensione (size)** indica quanto spazio effettivamente occupa il file. Le directory hanno una dimensione del file speciale contenente la lista di coppie (nomefile, inode_number) che solitamente è 4K (4096) ma più file ci sono e più la dimensione è grande (anche se sono tutti vuoti)

```
fra@Lenovo:~$ ls -l testfile
-rw-rw-r-- 1 fra fra 13 Mar 12 18:51 testfile
fra@Lenovo:~$ ls -l .
total 52
drwxr-xr-x 11 fra fra 4096 Mar 12 18:43 Desktop
drwxr-xr-x 2 fra fra 4096 Jan 3 00:06 Documents
drwxr-xr-x 3 fra fra 4096 Mar 12 11:12 Downloads
drwxr-xr-x 4 fra fra 4096 Jan 3 13:10 go
drwxr-xr-x 2 fra fra 4096 Jan 3 00:06 Music
drwxr-xr-x 2 fra fra 4096 Mar 12 17:15 Pictures
drwxrwxr-x 3 fra fra 4096 Jan 3 15:03 Postman
drwxr-xr-x 2 fra fra 4096 Jan 3 00:06 Public
drwx----- 3 fra fra 4096 Feb 28 15:04 snap
drwxr-xr-x 2 fra fra 4096 Jan 3 00:06 Templates
-rw-rw-r-- 1 fra fra 13 Mar 12 18:51 testfile
drwxr-xr-x 2 fra fra 4096 Jan 3 00:06 Videos
drwx----- 5 fra fra 4096 Feb 16 23:34 'VirtualBox VMs'
```

Un altro modo per ricevere **informazioni più dettagliate** di un file possiamo usare il comando

stat [-c format] {nomefile}

Es:

- **stat filename** restituisce varie informazioni
- **stat -c %B filename** restituisce la dimensione del blocco su disco che coincide con la dimensione di un settore di disco

```
fra@Lenovo:~$ stat testfile
File: testfile
Size: 13          Blocks: 8          IO Block: 4096   regular file
Device: 8,3      Inode: 5878743    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   fra)   Gid: ( 1000/   fra)
Access: 2026-03-12 18:53:10.867054101 +0100
Modify: 2026-03-12 18:51:00.299689153 +0100
Change: 2026-03-12 18:53:10.867054101 +0100
 Birth: 2026-03-12 18:44:21.480580441 +0100
fra@Lenovo:~$ stat -c %B testfile
512
```

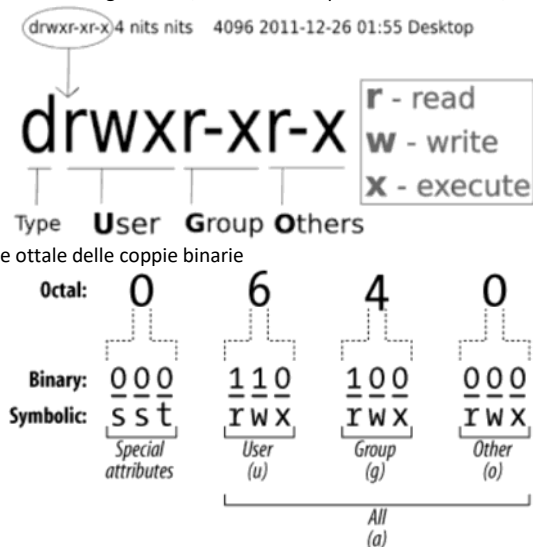
Permessi di Accesso

L'utente **proprietario** (chi crea il file) e il **gruppo proprietario** (gruppo dell'utente proprietario) possono accedere al file.

Inoltre il proprietario può definire i **permessi di accesso** per definire chi può **leggere, scrivere e eseguire il file**

I permessi sono definiti come una **stringa binaria da 12 cifre** ma normalmente si visualizza come:

- **Stringa di 10 caratteri**: dove il primo carattere è per l'attributo speciale e le 3 coppie di terne sono rispettivamente per **User, Group e Others (U, G, O)**
- Il tipo del file può essere "-" (file semplice), "d" (directory), "l" (link), ecc
- I permessi **rwX** sono la rappresentazione scritta della stringa binaria, es: se User ha permesso 000 ⇒ ---, 011 ⇒ -wx, 110 ⇒ rw-



- **Numero da 3 cifre**: rappresenta la conversione ottale delle coppie binarie

Permesso	Oct	significato
---	0	Non si può fare niente
--x	1	Si può settare come cwd (ma solo se il permesso c'è per tutte le directory nel path); si può anche "attraversare", se già se ne conosce il contenuto
-w-	2	Non si può fare niente
-wx	3	Come il permesso 7, ma non si può listare il contenuto
r--	4	solo listarne il contenuto (senza vedere attributi file), non può essere "attraversata"
r-x	5	Si può leggere (attributi compresi), settare come cwd ed attraversare; non è possibile cancellare o aggiungere file/directory

4	2	1	
0	-	-	- no permissions
1	-	-	x only execute
2	-	w	- only write
3	-	w	x write and execute
4	r	-	- only read
5	r	-	x read and execute

	4	2	1		--x	1	Si può settare come cwd (ma solo se il permesso c'è per tutte le directory nel path); si può anche "attraversare", se già se ne conosce il contenuto
0	-	-	-	no permissions	-w-	2	Non si può fare niente
1	-	-	x	only execute	-wx	3	Come il permesso 7, ma non si può listare il contenuto
2	-	w	-	only write	r--	4	solo listarne il contenuto (senza vedere attributi file), non può essere "attraversata"
3	-	w	x	write and execute	r-x	5	Si può leggere (attributi compresi), settare come cwd ed attraversare; non è possibile cancellare o aggiungere file/directory
4	r	-	-	only read	rwx	6	Come il permesso 4
5	r	-	x	read and execute		7	listare contenuto (attributi compresi), aggiungere file e directory, cancellare file contenuti in essa (anche senza avere il permesso di scrittura sul file! correggibile con lo sticky bit, vedere più avanti), cancellare directory contenute in essa (ma occorrono tutti i permessi su tali directory)
6	r	w	-	read and write			
7	r	w	x	read, write and execute			

Con **scrittura** (w) si intende: sovrascrivere, append (scrivere alla fine) e cancellare

Permessi Speciali

I permessi speciali possono essere applicati a file e directory e sono:

- **Set User ID o setuid o SUID (s)**: Usato solo per i **file eseguibili** che permette ad un file di essere eseguito con i **privilegi del suo creatore** e non di chi esegue il file. Se il proprietario è **root** il file è eseguito con i privilegi di root, indipendentemente da chi lo esegue (es comando passwd)
- **Set Group ID o setgid o SGID (s)**: Analogo a setuid ma per i gruppi, cioè i privilegi sono quelli del gruppo che è proprietario del file eseguibile. Può essere applicato anche ad una **directory**, così che ogni file lì dentro ha il gruppo della directory
- **Sticky Bit (t)**: applicato su **directory** (inutile su file) corregge il comportamento di **w+x** permettendo la **cancellazione di file solo se** si hanno permessi di scrittura su di essi.
Dato un utente U e un file f in una directory D che non appartiene a U né al gruppo di U, se **U cerca di cancellare f allora**:
 - **Senza Sticky Bit su D**: basta avere i permessi di scrittura su D (nel gruppo "other") per cancellare f, anche se non si hanno i permessi su f
 - **Con Sticky Bit**: è necessario avere i **permessi di scrittura su f** per cancellarlo

Visualizzazione Permessi

Il comando **ls -l nomefile** mostra solamente i permessi come stringa di 10 caratteri mentre **stat nomefile** mostra anche la visualizzazione numerica

I permessi **speciali** invece vengono visualizzati **al posto del bit di esecuzione**:

- Il **setuid** nella **terna user**
- Il **setgid** nella **terna group**
- Lo **sticky** nella **terna other**

Se il permesso di esecuzione è **attivo** allora la s/t saranno **minuscoli**, **altrimenti maiuscoli**

Es: se abbiamo **-rwsr-xr-T**

- -: è un file normale
- **rws** (user): **s** **minuscola** quindi **suid è attivo** e l'utente **può eseguire** il file
- **r-x** (group): c'è la x quindi **sgid non attivo** e l'utente **può eseguire** il file
- **r-T** (other): **t** **maiuscola** quindi **sticky è attivo** e l'utente **non può eseguire** il file

Modifica Permessi

Per modificare i permessi di un file/dir si usa il comando

```
chmod mode[, mode...] nomefile
```

- **Modalità numerica**: si usano 4 numeri tra 0 a 7 dove il primo numero indica setuid (4), setgid (2), sticky (1) e gli altri sono per user, group e other. Si possono fornire anche solo 3 numeri se si assume setuid, setgid, e sticky **settati a 0**
- **Modalità simbolica (lettere)**: per settare il mode usando i simboli si usa il formato `[ugoa][+=[perms...]`
 - **[ugoa]** rappresenta a **chi** viene attribuito il permesso (user, group, other, all)
 - **[+=[** rappresenta l'operazione da effettuare sul permesso impostato al target (aggiungi, rimuovi, imposta esattamente)
 - **[perms...]** può essere:
 - zero
 - una o più lettere dal set **{rwxXst}**
 - una lettera dal set **{ugo}**

Es:

- Rendere eseguibile per user e togliere tutti i permessi a group e other:


```
chmod 700 nomefile
chmod u+x,go-rwx nomefile
```
- Impostare i permessi di group agli stessi di quelli di user e dare a other solo la scrittura (togliendo eventuale lettura/esecuzione)


```
chmod 772 nomefile
chmod g=u,o=w nomefile
```
- Impostare l'esecuzione del file con i privilegi del proprietario (suid) e permettere solo al proprietario di cancellare il file da una cartella (sticky)


```
chmod 5772 nomefile
chmod u+s,o+t nomefile
```
- Dare a tutti il permesso di lettura


```
chmod 5776 nomefile
chmod a+r nomefile
```



```

fra@Lenovo:~$ ls -l testfile1
-rw-rw-r-- 1 fra fra 0 Mar 13 17:51 testfile1
fra@Lenovo:~$ chmod 700 testfile1
fra@Lenovo:~$ ls -l testfile1
-rwx----- 1 fra fra 0 Mar 13 17:51 testfile1
fra@Lenovo:~$ chmod 772 testfile1
fra@Lenovo:~$ ls -l testfile1
-rwxrwx-w- 1 fra fra 0 Mar 13 17:51 testfile1
fra@Lenovo:~$ chmod 5772 testfile1
fra@Lenovo:~$ ls -l testfile1
-rwsrwx-wT 1 fra fra 0 Mar 13 17:51 testfile1
fra@Lenovo:~$ chmod 5776 testfile1
fra@Lenovo:~$ ls -l testfile1
-rwsrwxrWT 1 fra fra 0 Mar 13 17:51 testfile1

fra@Lenovo:~$ ls -l testfile2
-rw-rw-r-- 1 fra fra 0 Mar 13 17:51 testfile2
fra@Lenovo:~$ chmod u+x,go-rwx testfile2
fra@Lenovo:~$ ls -l testfile2
-rwx----- 1 fra fra 0 Mar 13 17:51 testfile2
fra@Lenovo:~$ chmod g=u,o=w testfile2
fra@Lenovo:~$ ls -l testfile2
-rwxrwx-w- 1 fra fra 0 Mar 13 17:51 testfile2
fra@Lenovo:~$ chmod u+s,o+t testfile2
fra@Lenovo:~$ ls -l testfile2
-rwsrwx-wT 1 fra fra 0 Mar 13 17:51 testfile2
fra@Lenovo:~$ chmod a+r testfile2
fra@Lenovo:~$ ls -l testfile2
-rwsrwxrWT 1 fra fra 0 Mar 13 17:51 testfile2

```

chown (change owner) e chgrp (change group)

chmod serve a cambiare i **permessi** di un file, invece **chown** e **chgrp** servono a cambiare **a chi appartiene** il file

Entrambi **richiedono root** e:

- **chown** modifica sia il proprietario che il gruppo
chown [-R] proprietario {file}
Es: cambia proprietario e gruppo insieme (usando i due punti)
chown mario:studenti documento.pdf
- **chgrp** modifica solo il gruppo
chgrp [-R] gruppo {file}
Es:
chgrp studenti documento.pdf

Se si esegue su una directory con opzione **-R** si applica il comando a **tutte le sotto directory**

umask (User Mask)

umask [mode] è un comando che **"toglie"** i **permessi di default** per i nuovi file/dir create (la descrizione è nel man bash)

La maschera **sottrae i permessi** definiti (NOT(umask)).

Es se una dir ha permessi 777 e vogliamo togliere i permessi di write a group e others impostiamo umask 022 (777-022 = 755)

Spostamento, Modifica e Collegamento

cp (Copy)

Questo comando permette di **copiare** file/dir in altre cartelle

- ```
cp [-r] [-i] [-a] [-u] {filesorgenti} filedestinazione
```
- **-r (recursive)**: copia la cartella e i suoi contenuti
  - **-i (interactive)**: chiede conferma prima di sovrascrivere
  - **-a (archive)**: preserva tutto (permessi, date, link)
  - **-u (update)**: sovrascrive solo se l'mtime sorgente è più recente di quello di destinazione

### mv (Move) - Spostamento o Rinominazione

Questo comando permette di **spostare** file/dir in altre cartelle

- ```
mv [-i] [-u] [-f] {filesorgenti} filedestinazione
```
- **-i e -u**: come per **cp**
 - **-f (force)**: sovrascrive senza chiedere nulla (contrario di -i)

Per **rinominare** un file basta eseguire

```
mv nomefile nomenuovofile
```

rm (Remove)

Questo comando permette di **rimuovere** file/dir

- ```
rm [-f] [-i] [-r] {file}
```
- **-f e -r**: come per **mv** e **cp**
  - **-i**: chiede conferma per ogni file

### ln (Link) - Crea Collegamento

Questo comando permette di **creare un collegamento** ad un file/dir

Un **hard link (default)** ad un file è un nuovo file che punta a quel file **senza contenere in esso il suo contenuto**. Se il file originale viene eliminato il link continua a funzionare e i dati rimangono finché non vengono eliminati tutti gli hard link

- ```
ln [-s] sorgente [destinazione]
```
- **-s (symbolic)**: crea un **soft link**, scorciatoie basate sul percorso (tipo Windows). Se il file originale è rimosso il link non funziona più

touch - Crea/Aggiorna data

Questo comando **modifica il timestamp** di un file (o dir) e se un file **non esiste allora lo crea**

- ```
touch [-a] [-m] [-t timestamp] {file}
```
- **-a (access time)**: cambia solo l'ora di ultimo accesso
  - **-m (modification time)**: cambia solo l'ora di ultima modifica
  - **-t [timestamp]**: imposta una data specifica invece di quella attuale

## Informazioni su Disco

### du (Disk Usage) - Spazio Occupato da File

Questo comando mostra le **dimensioni dei file/dir** dati

- ```
du [-c] [-s] [-a] [-h] [--exclude=PATTERN] [files...]
```
- **-c (total)**: produce un totale complessivo alla fine della lista
 - **-s (summarize)**: mostra il totale di una cartella, senza elencare ogni file
 - **-a (all)**: mostra lo spazio occupato da ogni singolo file, non solo dalle directory
 - **-h (human-readable)**: mostra i num in KB, MB, GB

df (Disk Free) - Spazio Libero su Disco

Questo comando mostra le **dimensioni** e l'**attuale uso del FileSystem**

```
df [-h] [-l] [-i] [file]
```

- **-h**: come per du
- **-l (local)**: mostra solo i dischi fisici e non di rete
- **-i (inodes)**: mostra quanti inode sono ancora disponibili

dd (Data Duplicator)

Questo comando serve per creare/copiare file in modo elaborato a basso livello (bit a bit)

```
dd [opzioni]
```

Le **opzioni** sono una sequenza **variabile=valore**

Le variabili più importanti sono:

- **bs=BYTES**: dimensione di un singolo blocco in lettura/scrittura
- **count=N**: num. blocchi da copiare
- **convert=CONVS**: il valore specifica una conversione, per esempio di codifica
- **if=FILE**: file di input (se non dato, legge da tastiera)
- **of=FILEq**: file di output (se non dato, scrive su schermo)
- **skip=N**: salta N blocchi iniziali

Si usa anche per copiare file speciali che non si possono copiare con cp

È utile anche per:

- Cancellare i dati da un supporto di memoria
- Preparare un file ad essere formattato
- Copiare solo una parte di un file

mkfs (Make File System) - Formattare

Questo comando crea un **filesystem** su device e serve principalmente per formattare una partizione

```
mkfs [-t type fsoptions] device
```

- **type**: il tipo di fs (ext4, vfat, ntfs, ecc)
- **fsoptions**: può essere **ro** o **rw**
- **device**: device file in **/dev** oppure file regolare9

Es: `sudo mkfs -t ext4 /dev/sdb1` (formatta la partizione sdb1 in formato Linux ext4)

Processi

lunedì 16 marzo 2026 16:37

In linux abbiamo:

- **File:** descrivono/rappresentano le risorse
- **Processi:** permettono di elaborare dati e usare le risorse

file eseguibile in esecuzione = processo

Per lanciare un processo basta eseguire il file corrispondente da shell (o cliccare su un'icona nell'ambiente grafico)

Si creano processi usando i comandi già visti: `dd`, `ls`, `cat`, `cp`, `ln`, `useradd`, `group`, `sudo`, `su`, ecc (echo e `cd` vengono eseguiti dal processo di shell)

Ridirezione

Ridirezione a File

La **ridirezione** permette di deviare il **flusso dei dati** (input e output) da o verso file. Ogni file contiene **tre canali**:

- 1) **Standard Input (stdin - 0):** dove il programma **legge i dati** (default: tastiera)
- 2) **Standard output (stdout - 1):** dove il programma **invia i risultati** (default: schermo)
- 3) **Standard Error (stderr - 2):** dove il programma **invia i msg di errore** (default: schermo)

Possiamo usare i simboli `>`, `>>` e `<` per redirige l'output

- **> (sovrascrittura):** crea il file o ne cancella il contenuto per scrivere nuovi dati
`ls > lista.txt`
- **>> (append):** aggiungi i dati alla fine del file
`echo "nuova riga" >> log.txt`
- **< (lettura):** prendiamo i dati dal file
`sort < unsorted_list.txt`
- **<< (lettura multi-line):** manda più righe di testo

Ridirezionare Errori

Per la stampa degli errori possiamo:

- **2>file:** ridirezionare l'errore in un file
`ls > dirlist 2> errori.log`
- **2>&1:** ridirezionare l'errore verso lo standard output. L'ordine in cui viene usato in base al comando cambia la posizione di output
 - **Errore ridirezionato prima "comando > file":** l'errore viene ridirezionato allo standard output (1) di default quindi viene stampato su schermo. Il file conterrà solo l'output del comando
`2>&1 ls > dirlist`
 - **Errore ridirezionato dopo "comando > file":** lo standard output (1) viene ridirezionato al file quindi l'errore quando viene ridirezionato allo standard output verrà inviato sul file. Il file conterrà sia l'output standard che l'errore
`ls > dirlist 2>&1`

/dev/null è uno spazio dove i dati inviati spariscono nel nulla.

```
ls > /dev/null 2>&1
(così il comando non mostrerà né output né errori)
```

Pipe (Collegamento Comandi)

La **pipe (|)** non usa file ma collega l'uscita di un comando direttamente all'entrata di un altro

```
comando1 | comando2 | ... | comando n
```

Es: `ls -l | grep ".txt"` (l'elenco dei file passa a `grep`, che filtra solo i `.txt`)

Usando `|&` lo standard error viene ridirezionato sullo standard input del comando successivo

Rappresentazione dei Processi

PID (Process Identifier) e PCB (Process Control Block)

Il **PID** è un **identificatore univoco** di un processo (non possono esistere 2 processi con lo stesso PID) e una volta che il processo termina il suo PID viene liberato così da poter essere usato da un altro processo

Il **PCB** è una **struttura dati** che gestisce e controlla il processo (PCB unico per ogni processo). Tiene traccia di:

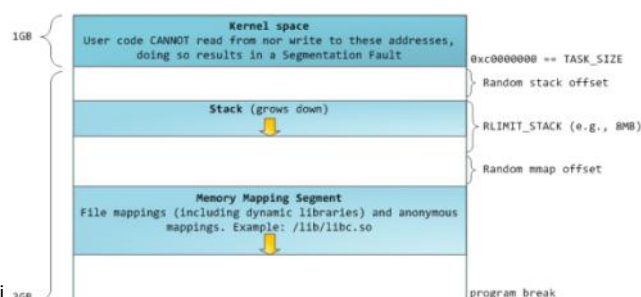
- **PID:** Process ID
- **PPID:** Parent Process ID
- **Real UID (RUID) e Real GID (RGID):** Real User/Group ID
(quando setto `SetUID/SetGID` di un file eseguibile, RUID/RGID è l'ID di chi lo esegue)
- **Effective UID (EUID) e Effective GID (EGID):** Effective User/Group ID (UID/GID assunto dal processo in esecuzione)
(quando setto `SetUID/SetGID` di un file eseguibile, EUID/EGID è l'ID del proprietario del file)
- **Saved UID e Saved GID:** Saved User/Group ID (UID/GID avuto prima dell'esecuzione di `SetUID`)
- **Current Working Directory**
- **Umask:** file mode creation mask
- **Nice:** priorità statica del processo

Aree di Memoria

Le **aree di memoria** sono **sezioni** logiche in cui viene diviso lo **spazio di indirizzamento virtuale** di un processo.

All'avvio di un processo gli viene assegnata una memoria strutturata in **cinque (o sei) zone principali**:

- **Text Segment:** istruzioni da eseguire
- **Data Segment:** memorizza **dati statici** (variabili globali e variabili locali statiche) e alcune costanti di ambiente. Si divide in:
 - **Data Segment Inizializzato:** contiene dati statici **inizializzati** nel codice (es `int count = 10`)
 - **Data Segment Non Inizializzato - BSS (Block Started by Segment):** contiene dati

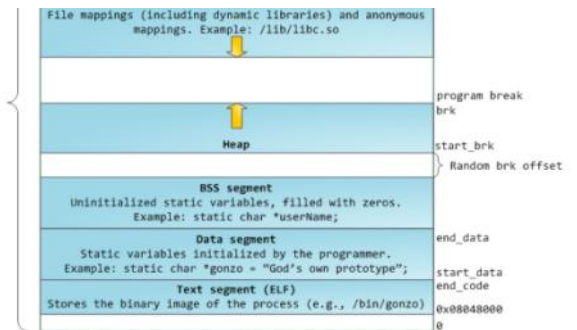


costanti di ambiente. Si divide in:

- **Data Segment Inizializzato**: contiene dati statici **inizializzati** nel codice (es `int count = 10`)
- **Data Segment Non Inizializzato - BSS (Block Started by Segment)**: contiene dati statici **non inizializzati**
- **Heap**: dati **dinamici** (allocati con `malloc` e simili)
- **Stack**: chiamate a funzioni con corrispondenti dati dinamici (variabili locali non static)
- **Memory Mapping Segment (MMS)**: librerie esterne dinamiche usate dal processo

Le aree di memoria possono essere **condivise**:

- Il text segment tra **più istanze dello stesso processo**
- 2 processi possono avere lo stesso BSS o Data Segment o MMS
- Lo **Stack** non è **mai condiviso**



Stato di un Processo

Un Processo può essere in 6 stati:

- **Running (R)**: in esecuzione su un processore
- **Runnable (R)**: Pronto per essere eseguito, in attesa che lo scheduler lo selezioni per l'esecuzione
- **(Interruptible) Sleep (S)**: in attesa di un qualche evento (es lettura blocchi su disco) e non può essere scelto dallo scheduler
- **Zombie (Z)**: processo terminato e le sue 6 aree di memoria non sono più in memoria; tuttavia il suo PCB viene mantenuto perché il processo padre non ha richiesto il suo "exit status"
- **Stopped (T)**: caso particolare di sleeping: ha ricevuto un segnale STOP e aspetta un segnale CONT per ripartire
- **Traced (t)**: esecuzione di debug o in attesa di un segnale (altro caso particolare di sleeping)
- **Uninterruptible sleep (D)**: come sleep ma facendo operazioni IO su dischi lenti e non può essere interrotto/ucciso

Esecuzione dei Processi

Un processo è in due **modalità di esecuzione**:

- **Foreground**: il comando legge input da tastiera e scrive su schermo. Finché non termina il prompt non viene restituito e non si possono usare altri comandi
 - **Background**: il comando non può leggere da tastiera ma può scrivere su schermo. Il prompt viene restituito e mentre il job è in background si possono dare altri comandi alla shell
- Per **eseguire un processo in background** si usa **&** alla fine del comando e nel terminale verrà restituito il suo PID e il num. del job (es `sleep 15s &`).
- Per **vedere i job in esecuzione** si usa `jobs [-l] [-p]` (-l stampa per ogni job il suo num., il PID, lo stato e il comando. -p stampa solo i PID)

Comandi bg e fg

Si può **interrompere** un comando avviato con **CTRL+Z** e si può risvegliare con:

- Il comando **bg** risveglia e avvia in background l'ultimo processo interrotto
bg n porta in background il processo interrotto n
- Il comando **fg** risveglia e porta in foreground l'ultimo processo interrotto
fg n porta in foreground il processo interrotto n

Si possono indicare job anche con:

- **%prefix**: dove **prefix** è la parte iniziale del comando
- **#+ o %%**: ultimo job mandato
- **%-**: penultimo job mandato

```
fra@Lenovo:~$ sleep 60s
^Z
[1]+  Stopped                  sleep 60s
fra@Lenovo:~$ bg
[1]+  sleep 60s &
fra@Lenovo:~$ sleep 10s
^Z
[2]+  Stopped                  sleep 10s
fra@Lenovo:~$ sleep 15s
^Z
[3]+  Stopped                  sleep 15s
fra@Lenovo:~$ bg
[3]+  sleep 15s &
fra@Lenovo:~$ fg 1
sleep 60s
^Z
[1]+  Stopped                  sleep 60s
fra@Lenovo:~$ bg 2
[2]-  sleep 10s &
[3]   Done                    sleep 15s
```

Processi in Esecuzione

Vedere i Processi

Comando ps

Il comando **ps** mostra le informazioni dei processi in esecuzione. Per ogni processo mostra PID, TTY, TIME e CMD

- `ps [-e] [-u {nomeutente, ...}] [-p {PID, ...}] [-f] [-l] [-o]`
- **Senza argomenti**: mostra i **processi dell'utente attuale** lanciati dalla shell corrente
 - **-e (every)**: mostra **tutti i processi** in esecuzione nel sistema (quindi anche processi degli altri utenti)
 - **-u {nomeutente, ...} (user)**: filtra in base all'utente dato
 - **-p {PID, ...} (PID)**: mostra info sul **processo specifico** con quel PID
 - **-f (full format)**: mostra il **formato esteso** dei processi
 - **UID**: nome utente che ha avviato il processo
 - **PPID**: PID del processo padre
 - **C**: uso della CPU
 - **STIME**: ora di inizio del processo
 - **CMD**: comando completo con tutti gli argomenti
 - **-l (long format)**: mostra il **formato ancora più esteso**
 - **F**: flag del processo (1 = processo "forkato" ma non eseguito, 4 = privilegi root, 5 = sia 1 che 4, 0 = nessuno dei precedenti)
 - **S**: stato del processo (S = Sleeping, R = Running, Z = Zombie)

- **PRI**: priorità (numero alto = priorità minore)
- **NI**: valore "nice"
- **ADDR**: indirizzo in mem del processo, mostrato (senza val) solo per compatibilità (-y -l toglie questo campo)
- **SZ**: dimensione tot attuale del processo in num di pagine (6 aree di mem del processo) sia in memoria che su disco (mem virtuale)
- **WCHAN**: se processo in attesa o in sleep mostra la funzione kernel all'interno della quale si è fermato
- **-o {field, } (output personalizzato)**: permette di **decidere cosa stampare**
Es: ps -o pid,user,command

Comando top (ps interattivo)

Il comando **top** mostra i processi in modo interattivo e refreshando la lista dei processi ogni pochi secondi

- top [-b] [-n num] [-p {pid,}]
- **-b (batch mode)**: non accetta comandi interattivi ma refresha e basta
- **-n num**: esegue num refresh e poi finisce
- **-p {pid, }**: come per ps

```
fra@Lenovo:~$ ps
  PID TTY          TIME CMD
 201449 pts/5    00:00:00 bash
 206670 pts/5    00:00:00 ps
fra@Lenovo:~$ ps -p 201449 -f
UID        PID     PPID  C  STIME TTY          TIME CMD
fra      201449   30139  0   12:04 pts/5    00:00:00 bash
fra@Lenovo:~$ ps -l
F S  UID        PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000    201449   30139  0   80   0  -   3810 do_wai pts/5    00:00:00 bash
4 R   1000    206716 201449 99   80   0  -   3482 -      pts/5    00:00:00 ps
fra@Lenovo:~$ ps -e -u fra -o pid,euser,args
  PID EUSER      COMMAND
    1 root      /sbin/init splash
    2 root      [kthreadd]
    3 root      [pool workqueue_release]
    4 root      [kworker/R-rcu_gp]
    5 root      [kworker/R-sync_wq]
    6 root      [kworker/R-kvfree_rcu_reclaim]
    7 root      [kworker/R-slab_flushwq]
    8 root      [kworker/R-netns]
   11 root      [kworker/0:0H-events_highpri]
   13 root      [kworker/R-mm_percpu_wq]
   14 root      [ksoftirqd/0]
```

```
fra@Lenovo:~$ top -b -n1
top - 12:04:38 up 4 days, 23:40, 1 user, load average: 2,38, 1,60, 1,36
Tasks: 389 total, 1 running, 388 sleeping, 0 stopped, 0 zombie
%Cpu(s): 3,7 us, 5,2 sy, 0,0 ni, 91,1 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MiB Mem : 7287,1 total, 976,3 free, 5578,2 used, 1088,4 buff/cache
MiB Swap: 9216,0 total, 5748,7 free, 3467,3 used. 1708,9 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
180111 libvirt+ 20   0 7711788 2,8g 16416 S 36,4 39,2 34:16.41 qemu-sy+
2588 fra      20   0 5378900 169792 40960 S 9,1 2,3 27:19.18 cinnamon
5092 fra      20   0 49,2g 95044 52044 S 9,1 1,3 19:19.86 opera
5311 fra      20   0 1394,1g 159436 81984 S 9,1 2,1 1:05.20 opera
57105 fra     20   0 1391,9g 14336 13668 S 9,1 0,2 0:01.52 code
201722 fra     20   0 14512 5664 3500 R 9,1 0,1 0:00.02 top
    1 root      20   0 23264 10388 6668 S 0,0 0,1 0:05.41 systemd
    2 root      20   0 0 0 0 S 0,0 0,0 0:00.10 kthreadd
    3 root      20   0 0 0 0 S 0,0 0,0 0:00.00 pool wo+
    4 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
    5 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
    6 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
    7 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
    8 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
   11 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
   13 root      0 -20 0 0 0 I 0,0 0,0 0:00.00 kworker+
```

Invio Segnali ai Processi

Il comando **kill** è usato per **terminare i processi** attivi, inviando un segnale al processo per terminarlo

- kill [-l [signal]] [-signal] [-s signame] [-n signum] [pid...]
- **-l**: mostra la **lista dei segnali**; un segnale **signal** è identificato dal numero o dal nome con SIG o senza SIG
- Si può indicare il segnale da usare scrivendo il segnale intero (-signal), il suo numero (-n num oppure -num) o il suo nome (-s signame con o senza SIG):
kill -SIGKILL pid oppure **kill -9 pid** oppure **kill -s SIGKILL pid** oppure **kill -s KILL pid**

I segnali verranno usati solo se il **real user** del processo è lo stesso che **invia il segnale** (oppure se lo invia un superuser)

Un processo che riceve un segnale fa un'azione predefinita o una personalizzata

Alcuni segnali comuni sono:

- **SIGSTOP, SIGSTP**: sospende il processo (es: **CTRL+Z**)
- **SIGCONT**: continuazione di processi sospesi (es: **bg**)
- **SIGKILL, SIGINT**: terminazione dei processi (es: **CTRL+C**)
- **SIGUSR1 e SIGUSR2**: impostati per essere usati dall'utente per le proprie necessita (utile per inviare segnali tra processi)

Altri Comandi

Il comando **nice** permette di lanciare un comando con un valore di **niceness** dato.

Il **niceness** è un'addizione sulla priorità: se positivo, ne aumenta il valore (priorità decresce), altrimenti ne diminuisce il valore (priorità cresce)

- **nice [-n num] comando**: lancia il comando con **niceness num** (0 se non dato) che va da **-20** (max priorità) a **19** (min priorità)
- **nice** (senza argomenti): stampa il niceness di partenza

renice si usa su un processo **già in esecuzione** per cambiargli la priorità

renice priority {pid}

Es: renice 10 1234 (porta a 10 la priorità del processo 1234)

Il comando **strace**, usato per la diagnosi, **traccia tutte le System Call** di un programma

- **strace comando**: mostra le system call fatte dal comando
- **strace -p pid**: mostra le system call di un processo già avviato
- **-o filename**: ridireziona l'output su un file